

Dynamic read & write optimization with TurtleKV

Tony Astolfi
MathWorks
Tufts University

Vidya Silai
MathWorks

Darby Huye
Tufts University

Lan Liu
Tufts University

Raja R. Sambasivan
Tufts University

Johes Bater
Tufts University

ABSTRACT

High read and write performance is important for generic key-value stores, which are foundational to modern applications and databases. Yet, achieving high performance for mixed and dynamic workloads is challenging due to fundamental trade-offs between memory use and I/O for retrieval and updates. Past work emphasizes the trade-off between read- and write-optimization as expressed through primary data structure, in combination with read-memory trade-off mechanisms like caching and filtering. This raises re-tuning costs as optimal trade-off targets change, due to restructuring of stored data. We show that write-memory trade-off mechanisms are under-developed in current designs, and propose a new approach to *dynamic key-value store optimization* using a novel read-/write-balanced on-disk structure, the TurtleTree, and flexible read-memory & write-memory tuning knobs. We describe how the design of TurtleKV, our prototype, avoids in-memory bottlenecks to achieve high performance across a wide range of tuning parameters. When evaluated using YCSB, TurtleKV matches state-of-the-art SplinterDB for inserts, and is 5 $\times$ /12 $\times$ faster than RockDB/WiredTiger. In mixed workloads, TurtleKV is 16-25% faster than SplinterDB, >4 $\times$ RocksDB, and 3-6 $\times$ WiredTiger. TurtleKV is 2-9 $\times$ faster than the others for point-query workloads, and has the best scan performance of the write-optimized systems tested.

1 INTRODUCTION

Today the builders of complex, cloud-based applications and systems have many different key-value stores to choose from, each with their own biases and trade-offs (e.g., read-optimized vs write-optimized; online vs analytic; in- vs out-of-memory; ordered vs hashed; domain-specific vs general-purpose; etc.) [9, 17, 34, 40, 53–56]. These are frequently difficult to tune to accommodate evolving requirements after a system is built [1, 2, 4, 8, 16, 33, 39, 50]. As a consequence, we see a proliferation of multi-stores with complex and expensive logic for synchronization, data transfer, and invariant maintenance [26, 32, 42, 47, 49]. Engineering effort must be spent multiple times for development, debugging, optimization, and maintenance of each storage engine.

One would not be without reason believing that a multitude of storage engines is inevitable. The RUM Conjecture [10] posits a fundamental, three-way trade-off in the design of data storage/access methods between the resource overhead of reads (R), updates/writes (U), and memory/storage (M). If correct, this might seem to imply that the divergence of requirements for different workloads and applications means key-value stores inhabit a problem space where “one size” does not, and cannot, fit all [37]. Our work assumes the correctness of the RUM Conjecture as its starting point, but seeks to

challenge this implication and propose an alternative approach to key-value storage engine design in the face of inherent trade-offs.

The notion of a three-way trade-off in the RUM Conjecture means that at least two trade-off selection mechanisms, or *tuning knobs*, must be utilized to fully cover the space of optimal possibilities; e.g. reads-vs-writes (RW), reads-vs-memory (RM), or writes-vs-memory (WM). Our key insight is that these basic trade-offs have important differences, and that these differences make some inherently better suited for use as tuning knobs. For example, if a suboptimal tuning point is selected for the read-memory trade-off, it is likely easy to correct because of the mechanisms involved: simply modify the cache size to decrease either the I/O cost of queries or the memory footprint of the system at the expense of the other.

In contrast, if a system is improperly tuned with respect to the read-write trade-off, things aren’t so simple. This is because read-write tuning is largely accomplished via selection of primary data structure, which affects implementation (code) and the invariants of stored data, both of which are expensive to modify post-facto. While some dictionary data structures do support parameter-based read-write trade-off tuning (e.g., compaction policy in LSM-trees with adjustable fanout [48, 51]), when analyzed closely and evaluated empirically, we find the mechanisms in common use to be sub-optimal. In any case, even if the primary data structure of a system does support smooth, optimal tuning between read and write optimization (e.g. B⁺-trees’ ϵ parameter [13, 15]), bias is still encoded at the storage level, making re-tuning expensive.

Since write-optimization is very important to certain workloads [16], write-memory (WM) tuning must be supported as a first-class feature if we wish to avoid the downsides of RW tuning bias in stored data structures. To do this it is clearly necessary, though insufficient, to simply expose a tuning knob. The choice of data structure and the overall design of the system must work together to maximize the effectiveness of this trade-off. A good example of how data structures can anticipate a resource trade-off (RM) in a system design context is the pervasive use of B⁺-trees in preference to B-trees [11, 21]. By storing all key-value records at the bottom level of the tree, B⁺-trees offer increased branching factor and the possibility for all interior nodes to be cached in memory, reducing read amplification to $O(1)$. A key idea of the work presented here is to apply a similar principle in data structure selection to enhance the effectiveness of the write-memory (WM) trade-off, elevating it to first-class status in key-value system design.

We present TurtleKV, a proof-of-concept for a new approach to key-value storage oriented around RM and WM tuning knobs which dynamically navigate the RUM trade-off space without having to change the structure of stored data. TurtleKV is built around a novel data structure, the TurtleTree, which offers two important

properties: 1. it achieves a neutral, balanced compromise between the extremes of read- and write-optimized external memory dictionaries, and 2. it affords a smooth trade-off between memory usage and write amplification which is simultaneously efficient, scale-independent (in terms of data size N), conceptually simple, and wide-ranged in effect. TurtleKV implements a system architecture which is similar at a high level to conventional LSM-based key-value stores, with the addition of several crucial optimizations to eliminate in-memory bottlenecks so that reductions in write-amplification translate to increased throughput. Because TurtleKV starts from a fundamentally stronger premise, it avoids many of the complex workarounds and features which tend to creep into modern key-value stores in order to try to mitigate their intrinsic limitations. As a result, we show that TurtleKV outperforms today’s key-value engines, while offering superior tunability, with a modest code base of similar or lesser complexity.

In this work we demonstrate that write-vs-memory tuning mechanisms are often not given first-class status in the design of current key-value stores (§2.2). For example, while RocksDB [3], one of the most popular key-value storage engines today, supports runtime adjustment of write buffer size [4], and our experiments show this parameter is highly effective at spending memory to reduce write amplification, doing so yields very little benefit in terms of increased end-to-end performance. WiredTiger [7], another popular storage engine on the opposite end of the read-write spectrum, also supports WM tuning, and even demonstrates end-to-end benefit in terms of increased insert throughput over wide range of tuning points. However, WiredTiger’s B^+ -tree based design offers little benefit at low-to-moderate memory allocations due to the inherent limitations of the data structure [38]. Finally, we find that SplinterDB [5, 22, 23] which is optimized for high concurrency and fast storage devices, is able to take advantage of higher memory allocations to reduce write amplification and average insertion latency for randomized insertion workloads. However, since SplinterDB does not offer any way to express a preference for optimization target (i.e., use cache memory to optimize writes vs reads), in mixed workloads the optimization effect of higher cache size may become unpredictable, as the dueling concerns of updates and queries vie for limited memory resources.

The contributions of this work are:

- (1) An analysis of how current key-value stores and their data structures fall short of fully exploiting the potential of the RUM space, especially with respect to the *write-memory trade-off*
- (2) A novel external memory dictionary type, the *TurtleTree*, within a previously unidentified class of data structures, $B^{\epsilon+}$ -trees, which combines ideas from both read- and write-optimized dictionary types
- (3) A key-value storage engine design, *TurtleKV*, which uses Turtle-Trees to offer simple and smooth trade-off tuning for a wide range of resource allocations
- (4) An experimental evaluation of TurtleKV as compared to SOTA key-value stores WiredTiger, RocksDB, and SplinterDB

2 READ, UPDATE, MEMORY: PICK TWO

The RUM Conjecture [10] states that if one holds any of the three resource overheads (read, update/write, and memory/space) constant,

the remaining two are placed in a zero-sum trade-off relationship. Geometrically, this can be visualized, as in Figure 1, by a three dimensional resource space, where points represent the actual resource cost of a concrete data access method with respect to a particular workload. The notion of a three-way trade-off, in this conceptualization, is represented by a two dimensional surface or manifold embedded within the larger-dimensional space (dark gray). Fixed resource commitments can be thought of as two dimensional slices of this surface orthogonal to the axis of the fixed resource, with the resulting intersection defining the trade-off curve between the remaining two resources.

Navigating the optimal two-dimensional subset of the RUM space requires the use of exactly two mechanisms, or tuning knobs, which serve as a coordinate system within the sub-space. In this section we analyze our problem space by considering three tuning knob candidates: the read-write trade-off (RW), read-memory trade-off (RM), and write-memory trade-off (WM). These are explored through the perspectives of data structure (§2.1) and system design (§2.2). We conclude with a sketch of the key ideas behind TurtleKV (§2.3).

2.1 Data Structures: Trade-off Tuning

The most consequential design decision for a key-value storage engine is its choice of primary data structure. Here we review the major categories of external memory dictionary, grouped by their RW trade-off optimization target (read-optimized, write-optimized, and hybrid/balanced). We highlight the essential insights of each data structure, how each approaches trade-off tuning, and their inherent limitations.

2.1.1 Read-Optimized (B-trees). B-trees[21] provide optimal [15, 59] read performance and can utilize memory to reduce the number of reads required to process queries by caching pages containing tree-nodes (RM trade-off). B^+ -trees, the most commonly implemented B-tree variant, exploit this mechanism by placing key-value data in the leaves, allowing all but the bottom layer of the tree to be cached in main memory so that queries require only a single read.

State-of-the-art approaches to reducing the write cost of updates in B-trees employ the use of a write-ahead log (WAL) to ensure durability of updates, plus a scheme for applying updates to the tree in memory (WM trade-off). We refer to these as *Log Structured B-trees*. Their goal is to defer rewriting pages containing updated keys, in hopes of collecting enough updates per page to amortize I/O. Bw-trees [46] do this through the use of delta records applied in-memory to the main tree using lock-/latch-free techniques. The

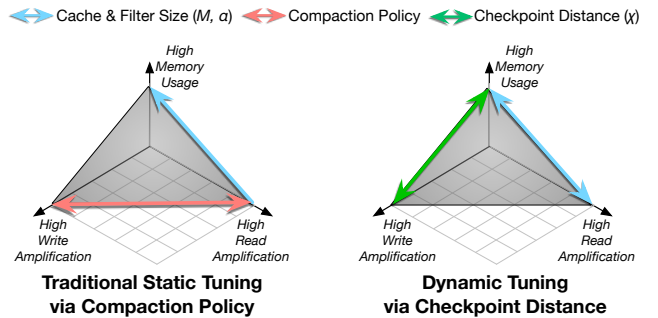


Figure 1: RUM as a three-dimensional resource space

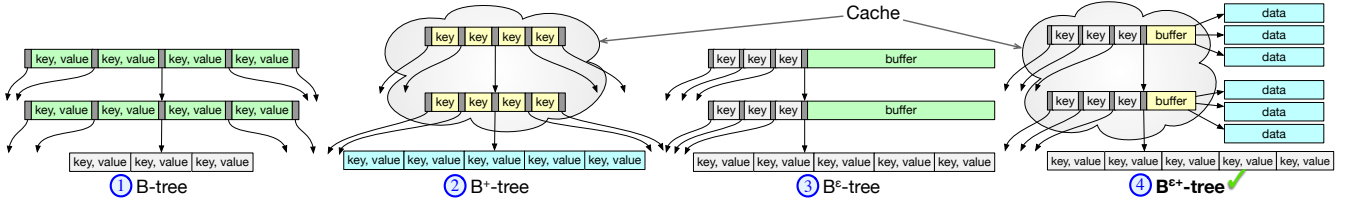


Figure 2: B^ϵ -tree $\rightarrow$ $B^{\epsilon+}$ -tree structural transformation

closely related Bf-tree [35] uses an alternate strategy of caching and updating sub-page slices (*mini-pages*) instead of full pages, which has additional benefits in terms of lowering the memory cost of caching “hot” records.

Bw-trees and Bf-trees achieve good write efficiency either when memory is plentiful or when updates are clustered in a few sub-ranges of the key space. In the general case, however, Log Structured B-trees force a sub-optimal trade-off between memory and write amplification, especially as the total data size N grows with respect to available memory M [38].

2.1.2 Write-Optimized (LSM-trees). Log-Structured Merge (LSM) trees [51] are a popular write-optimized alternative to B-trees. By deferring the merge (or *compaction*) of new updates with existing data, they maximize the amount of effective work achieved by each write operation. This results in a dramatic reduction in write amplification, but presents two challenges: first, it increases read costs (and storage footprint) by introducing the possibility that updates for a given key may be stored in multiple locations. Second, since the increasingly high bandwidth of modern storage devices favors performing merges in large sequential runs to maximize I/O utilization, update costs in LSM-trees tend to be *amortized* costs; i.e. tail latency can be quite high. This motivates the need for effective de-amortized, or incremental, approaches to compaction.

Page caching is less effective as an RM tuning mechanism for LSM-trees than B-trees because there is potentially much more data to cache. In addition to caching, two primary tuning mechanisms are employed by LSM-tree systems: compaction policy and approximate membership query (AMQ) filtering. Compaction policies divide into *greedy* (level-tiering) and *lazy* (size-tiering). In a level-tiered LSM-tree, write cost is increased by repeatedly merging data at the same level (up to F times) to reduce future read cost. In a size-tiered LSM-tree, a tunable number T of key duplicates are allowed to co-exist at a given level before their eventual merge into a larger level. Compaction policy thus alters the invariants of stored data to achieve a RW trade-off mechanism.

Approximate member query (AMQ) filters, e.g. Bloom Filters [14], are an RM trade-off mechanism that reduces the cost of point queries by having a dedicated filter (or part of a shared filter [23]) devoted to each location where a key might reside. AMQ filters offer a tuning parameter α^1 , the false-positive rate, which trades size against accuracy.² To answer a point query for key k , the filters for all locations S where k might reside are consulted before performing any I/O. This rules out, with probability $\prod_{i \in S} (1 - \alpha_i)$, locations where the query will fail, preventing excess reads.

¹Also frequently named as ϵ

²A filter must always answer accurately if a query key is not in its set, but it may give a false positive result at rate α . For lower α , more bits per key are required.

If filters are used to maintain a target read cost in a size-tiered LSM-tree (with T tiers per level), the total size of the filters must scale in proportion to T , since the total number of false-positives for a query is the sum of false-positives for all filters that pertain to a given key [28]. As T increases, read amplification increases while write amplification goes down; as filter bits-per-key increases, memory usage rises while read amplification falls due to lower false-positive rate α . It is theoretically possible therefore to coordinate the motion of these two parameters (T, α) to target a fixed level of read amplification, while writes are effectively traded against memory (WM) using filter size-vs-accuracy as an intermediary. We refer to this strategy as *filter arbitrage*.

2.1.3 Hybrid-Optimized (B^ϵ -trees). B^ϵ -trees [13, 15] are a tunable external memory dictionary, able to trade reads against writes. The basic B-tree structure is used as the foundation for a B^ϵ -tree. The ϵ parameter varies the portion of a node page used for pivots, which optimize for reading, and update buffering, which optimizes for writing. At either extreme of their tuning range $\epsilon \in 0..1$, B^ϵ -trees simply degrade into the equivalent of a B-tree or LSM-tree. At $\epsilon = \frac{1}{2}$, the branching factor is $\sqrt{B}$, making tree height twice that of an equivalent B-tree. In this configuration, updates can be *flushed* down one level at a time from the root in batches of size $\Theta(\sqrt{B})$, dramatically lowering the worst-case write cost and providing a good balanced tuning point.

One disadvantage of traditional B^ϵ -trees, where updates are buffered inline within the node page (Figure 2, ③), is that, for any given value of ϵ , increasing the page size B to optimize writes increases the I/O and/or memory cost of reading/caching a node. Similarly, decreasing ϵ to decrease the branching factor and increase the buffer size increases the height of the tree, which raises the cost of reading/caching a node-path to any given key.

Size-Tiered B^ϵ -trees (STB $^\epsilon$ -trees), used by SplinterDB [22, 23], are a recent re-imagining of the B^ϵ -tree designed to close the “bandwidth gap” between the capabilities of fast NVMe SSDs and traditional key-value storage engine designs. The main structural modification in STB $^\epsilon$ -trees is to add a level of indirection between the tree of interior node pages, or *trunk*, and buffered data, or *branches* (Figure 2, ③ $\rightarrow$ ④). This separation allows buffered updates to be flushed in larger batches than a traditional B^ϵ -tree, enabling high I/O bandwidth utilization. It also enables flushes to be decoupled from compactions since only the branch references need to be changed, enabling SplinterDB’s novel *flush-then-compact* policy. Flush-then-compact has the added benefit of allowing updates with skewed key distributions to be compacted at a lower level of the tree, reducing write amplification for “hot” records. However, due to their lazy compaction policy, STB $^\epsilon$ -trees must do more work to perform scan queries, which hurts performance.

2.2 System Design: Write-Memory Tuning

We now shift our attention from data structures to the concerns of system design, to explore how well current key-value storage engines support write-vs-memory (WM) trade-off tuning. We present three case studies of systems spanning the read-write (RW) spectrum covered in §2.1: WiredTiger, RocksDB, and SplinterDB. All are high-performance key-value storage engines optimized for concurrent workloads run on hardware with multiple processor cores and solid-state storage devices. RocksDB and WiredTiger in particular are heavily relied upon in real-world systems, and have been the beneficiaries of a large investment in engineering effort. The main differentiator in these systems is the choice of primary data structure: WiredTiger uses B⁺-Trees³, RocksDB is based on LSM-Trees, and SplinterDB introduces the novel STB^ε-tree.

An investigation into the limitations of how each of these systems support WM trade-off tuning elucidates some of the central issues and provides motivation for the design of our prototype, TurtleKV. Specifically, we present three findings: first, a system’s ability to navigate the WM trade-off is highly dependent on the choice of primary data structure; second, the ability to reduce write amplification does not always translate into increased real-world performance; and third, supporting I/O vs memory trade-offs in general does not imply the ability to target specific tuning points in terms of memory-based read- or write-optimization.

We summarize the key results of these case studies in Table 1.

2.2.1 WiredTiger. WiredTiger is the default storage engine for MongoDB. It is designed to support high levels of concurrency for transactional key-value workloads. To accomplish this, it uses lock/latch-free algorithms and multi-version concurrency control (avoiding update-in-place) to implement its primary data structure, a B⁺-tree.

Updates to the B⁺-tree in WiredTiger take place in “dirty” buffers pinned to its in-memory page cache, where new versions of existing pages (or entirely new pages) are built. Dirty pages are flushed to durable storage in response to a checkpoint request or when their total size exceeds a runtime-configurable threshold⁴. Write-ahead logging can be enabled for durability of record-level updates in between dirty page flush triggers. Thus, WiredTiger supports dynamic write-memory (WM) trade-off tuning through its dirty page eviction (write-back) caching mechanism.

Figure 3a shows how WiredTiger responds to increased memory allocation M for dirty page buffers, running a workload with

³WiredTiger also supports LSM-based indexing, but it is not the default.

⁴eviction_dirty_target for background threads, eviction_dirty_trigger for application/foreground threads.

Table 1: The three key requirements for dynamic write-memory trade-off tuning in key-value storage engine design.

	WiredTiger	RocksDB	SplinterDB	TurtleKV
Data Structure supports efficient WM trade-off?	✗	✓	✓	✓
System Design has no in-memory bottlenecks?	✓	✗	✓	✓
First-class write-memory trade-off tuning knob?	✓	✓	✗	✓

uniformly distributed keys inserted in random order. Here we see that average insert latency tracks with write amplification through the entire dynamic range of the resource trade-off, which is good. Reductions in write amplification factor (WAF), however, are minimal for $16\text{MB} \leq M \leq 500\text{MB}$; at higher memory allocations, WAF reduction accelerates. This is because, for random (non-skewed) key distributions, the expected benefit of buffering in B⁺-Trees is related to the total dictionary size N . If each page can hold B items, then the expected write amplification W_x for record x is $O(\max(1, \min(\frac{N}{M}, B)))$. Note, $W_x \in [1..B]$, since each record must be written at least once (1), and in the worst case it is re-written every time each other record in the same page is written (B).

2.2.2 RocksDB. RocksDB is used at Meta as the primary storage engine of ZippyDB (a distributed key-value storage service developed by and used within Meta), UDB (a MySQL storage layer for social graph data) [16], and by countless other open source projects and commercial systems.

RocksDB’s Tuning Guide [4] presents a formidable set of user-selectable policies and tuning knobs which optimize for various use cases. We identify two such knobs which appear to have the greatest impact on the WM trade-off: the size $\overline{M}_w$ of an in-memory table (*MemTable*)⁵, and the number of MemTables $|M_w|$ ⁶ collected in memory before being flushed to the first level of external storage (L_0). We find that varying $\overline{M}_w$ with fixed $|M_w|$ is superior to varying $|M_w|$ for fixed size $\overline{M}_w$, due to the degradation of query performance as the number of MemTables increases; both approaches offer the same benefit in write-optimization.

RocksDB’s default compaction strategy (leveled) uses incremental merging, in which levels are segmented into fixed-size sorted string table (SSTable) files. When the total size of all segments within a level L_i grows too large, an SSTable (of size $\overline{M}_w$) within L_i is selected to be *compacted* with data in the next larger/older level L_{i+1} . The compaction merges the single segment from L_i with all segments in L_{i+1} whose key range overlaps, producing a new sequence of SSTable segments whose keys are ordered and unique. The total write amplification W_x for a given record x can be calculated as the number of times F (for *fanout*) the record is merged into a segment at a given level, times the number of levels in the LSM⁷: $W_x = F \cdot \log_F(N/\overline{M}_w)$. Increasing MemTable size M therefore spends memory to reduce write amplification, independent of total size N : $\Delta W_x = -O(\log \overline{M}_w)$.

Figure 3b shows the result of repeatedly running a uniform random insertion workload against an empty RocksDB database, while increasing $\overline{M}_w$ from 16MB up to 4GB. When $16\text{MB} \leq \overline{M}_w \leq 1\text{GB}$, we see the predicted logarithmic reduction in write amplification (x-axis is log scale). However, for MemTable/SSTable sizes over 1GB, the write optimization effect flattens out, since the inserted/flushed data never escapes L_0 before the workload completes⁸. Unfortunately this reduction in write volume does not translate into lower

⁵rocksdb.options.write_buffer_size

⁶rocksdb.options.min_write_buffer_number_to_merge, rocksdb.options.max_write_buffer_number

⁷The reason there are $\log_F(N/M) = \log_F N - \log_F M$ levels for data set size N is that the smallest level’s size is always at least M ; one can think of the first $\log_F M$ merges of a record as happening in main memory, inside a MemTable.

⁸Key ranges of SSTable files in L_0 are allowed to overlap, so as not to stall MemTable flushes; at least 4 files must be present in L_0 to trigger a compaction to L_1 .

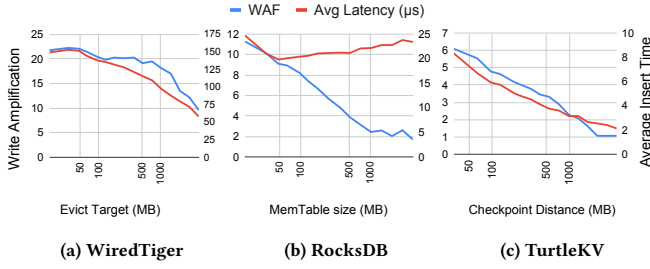


Figure 3: Write-Memory Trade-off Scaling (N=100M×128B)

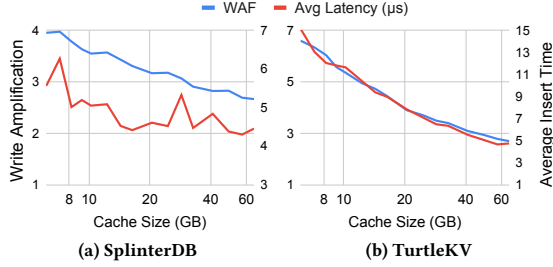


Figure 4: Write/Cache Size Trade-off Scaling (N=500M×128B)
Note: SplinterDB sacrifices read performance for faster ingest/updates (see Figure 8 & Figure 10, §5.2)

average insert latencies beyond RocksDB’s default value of $\overline{M}_w = 64\text{MB}$. This underscores the necessity of optimizing main-memory costs (CPU, memory bandwidth, and internal synchronization bottlenecks) if one targets I/O reduction as a tuning mechanism.

2.2.3 SplinterDB. SplinterDB largely adopts the strategy of selecting a single tuning point which attempts to offer excellent read and write efficiency by structuring per-node buffer data pages as a single-level, size-tiered LSM-tree-like structure, where each sorted run (or *branch*) is stored as a statically built B^+ -tree. Quotient Maplets, a variant of Quotient Filters [52], are used in this design to route point queries to the branches/tiers which may contain a given key. SplinterDB’s STB^ϵ -trees have configurable branching factor, which is tied to the number of tiers per level T . Quotient filter bit rate may also be tuned, and thus STB^ϵ -trees theoretically support a degree of write-memory tuning via filter arbitrage.

When we attempt to measure the effectiveness of these tuning parameters in practice, we observe minimal effects from varying fanout, filter size, and memtable size in SplinterDB. In fact the only parameter we observe to have a significant effect on end-to-end performance and I/O amplification is cache size, as shown in Figure 4a. We increased the data set size to 500M×128B for this test to show more of the tuning range, since at N=100M WAF was nearly optimal (≈ 1), even at small cache size. SplinterDB’s size-tiered lazy compaction policy achieves very low write amplification at small cache sizes, and for the most part average insert latency tracks with WAF. However, we find SplinterDB’s dynamic range of WM tuning to be limited, RM and WM cannot be independently targeted, and the task-parallel (§4.2.2) compaction strategy tends to generate a high variance in insert latencies.

2.2.4 TurtleKV. For comparison, we show TurtleKV’s results for both N=100M write-buffer size scaling and N=500M cache size-scaling workloads in Figures 3c and 4b respectively. These results

show that TurtleKV’s primary data structure, the TurtleTree, efficiently reduces write cost at a rate of $-O(\log(M))$, where M is the memory spent on write-optimization, like level-tiered LSM-tree-based RocksDB. Furthermore, because of the system design ideas elaborated in the following sections, TurtleKV is able to turn WAF reduction into higher performance over a very wide tuning parameter range, similar to WiredTiger. When configured for write optimization, TurtleKV keeps pace with SOTA engine SplinterDB, and because TurtleKV’s data-parallel strategy for in-memory compactions is able to achieve even load balancing across processors, its scaling curve is much smoother than SplinterDB’s (Figure 4).

2.3 Towards Dynamic Tunability: Key Ideas

Table 1 expresses the three high-level requirements which drive the design of TurtleKV. In this section we provide a brief sketch of the key ideas which address each requirement. These ideas will be elaborated in the following sections.

2.3.1 Data Structure With Efficient WM Trade-off. We start with the STB^ϵ -tree, observing that its desirable properties derive from the trunk/branch separation, whereas the properties we would like to change derive from their compaction and flush policies. We coin the term $B^{\epsilon+}$ -tree to describe the class of dictionaries which retain the trunk/branch separation, but which leave open the questions of flush/compaction policy and how (if at all) branches are structured within their referent trunk node(s) (Figure 2). Our first key idea is to instantiate the $B^{\epsilon+}$ -tree with a balanced, *level-tiered* compaction policy which organizes branches into sub-levels within a trunk node, as shown in Figure 5a. This structure we name as the TurtleTree.

2.3.2 System Design With No In-Memory Bottlenecks. TurtleKV adopts a design similar to LSM-tree-based systems like RocksDB, with a write-ahead log (WAL), in-memory component (MemTable), and log-structured on-disk component (Checkpoint, a TurtleTree). The primary bottleneck in this design comes from the $\log(N)$ amplification when merging MemTables into the current Checkpoint. We observe that the most CPU-intensive part of this process is the key comparisons for level-to-level merging, and the most memory-bandwidth-intensive part is the copying of key-value data to serialize compacted data at its new location. Our second key idea is to use data-parallel algorithms for level merging, and fork/join style multi-threaded leaf page serialization for Checkpoint writes.

2.3.3 First-Class Write-Memory (WM) Tuning Knob. Inspired by our RocksDB case study, we identify MemTable size $\overline{M}_w$ as our WM tuning knob. This defines the distance, in total bytes of updates, between Checkpoints. To ensure that increasing checkpoint distance does not degrade in-memory query performance, we build a single large MemTable between checkpoint updates. To keep tail latencies under control, and to avoid affecting the on-disk structure of checkpoints, we break finalized MemTables into leaf-page sized batches by doing a key-order scan, applying the batches to the Checkpoint TurtleTree one at a time in-cache until the MemTable is drained. This is our third key idea, which we call checkpoint distance tuning with Big MemTables.

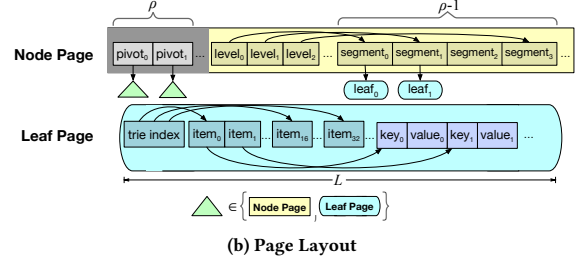
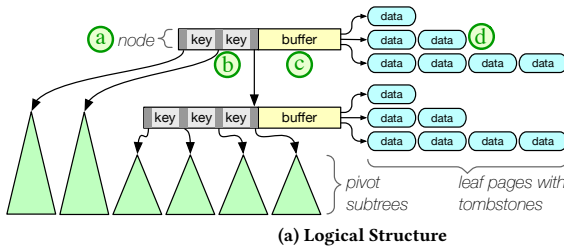


Figure 5: The TurtleTree data structure.

3 TURTLETREES

In this section we describe the TurtleTree data structure, its core algorithms (batch update, get, and scan), and how it leverages checkpoint distance as a WM trade-off tuning mechanism.

3.1 TurtleTree Structure

TurtleTrees are B^{e+} -trees (§2.3) whose per-node update buffer (c) is organized into a fixed number of levels (d), as in Figure 5a. Figure 5b shows the layout of TurtleTree node and leaf pages.

3.1.1 Leaf Pages. Leaf pages contain a sorted list of key-value pairs. To achieve high I/O utilization during updates and scans, leaf pages are sized several orders of magnitude larger than interior nodes.⁹ In order to guarantee low latencies during point queries, leaf pages are internally structured to allow small slices, or *sharded views*, to be cached and re-used to find queried keys. Leaves also include a cache-oblivious, byte-addressed binary trie index at the front of the page. This index contains every k^{th} key from the leaf, $k \in \{1..32\}$. The trie is made cache-oblivious by laying out nodes in Van Emde Boas order [12] to maximize locality of reference along any given query path, minimizing CPU cache misses for in-memory queries.

3.1.2 Node Pages. Each (interior) node (a) is split into roughly equal halves,¹⁰ with one half devoted to pivots (dark gray (b)) and the other to the update buffer (yellow (c)). Within a node page, the update buffer region contains pointers to *segments*, which are stored as leaf pages with the addition of tombstone records for deleted keys. The segments in a buffer are stratified into levels of exponentially increasing size. Keys are required to be *in-order* and *unique* within a level. The first level contains at most a single segment, the second level at most two, the third level four, then eight, etc. All levels start out *vacant* and can be in either a *vacant* or *occupied* state. For a given number of pivots ρ in a node, the number of levels is limited to $\lceil \log_2 \rho \rceil$, and the total number of segments in all levels of the buffer limited to at most $\rho - 1$.

Node buffers maintain certain metadata to speed up various operations. For each segment, we store a bit set *activePivots* of the pivots addressed by at least one active key in the segment,¹¹ and a compressed sparse array *flushedPivots* of key index upper bound values for each pivot to which keys from the segment have been flushed. When keys are removed from a level as part of a flush operation, the segment pages are not modified; instead, the flushed upper bound array is updated for that pivot so it points to

the segment's first non-flushed key in the flushed pivot key range. When all keys addressed to a given pivot have been flushed from a segment, the segment's active bit is cleared for that pivot, and the flushed upper bound is reset to zero. If this causes the active set to become empty, the segment is removed from the node, possibly allowing its storage to be garbage-collected.¹²

Node pages also maintain a count of the number of bytes of buffered updates in the key range of each pivot. This is used to decide when and how to perform flush operations. The default policy is to flush one leaf-sized batch after each batch insert, to the pivot with the highest *pendingBytes* count, if possible.

3.2 TurtleTree Operations

3.2.1 Batch Updates. TurtleTrees are designed to support efficient batched updates; individual key modifications can be logged/buffered outside the tree and applied as a batch once enough have accumulated. *TurtleTrees do not directly implement single-key Put or Delete.*

When the tree is empty, or consists of a single leaf, most operations are trivial. Batch update on a leaf consists of doing a merge/compact on the new and old data; if the result is small enough to fit in a leaf page, then the tree remains a single leaf. If not, then a new node is created to parent the two leaves. The root node is the only node in the tree allowed to have an arbitrarily small number of child pivots, so long as it is at least 2.

To perform a batch update on a node, we first insert the incoming batch into the node's buffer. To illustrate how this works, we work through a simple example, shown in Figure 6. In this example, we process four batches, containing keys $\langle 1, 7, 10 \rangle$, $\langle 0, 4, 5 \rangle$, $\langle 2, 8, 11 \rangle$, and $\langle 3, 6, 9 \rangle$ respectively. The node's buffer is initially in an *empty* state, i.e. all levels are vacant.

When the first batch $\langle 1, 7, 10 \rangle$ is inserted, we look at the first level of the buffer to see if it is vacant or occupied. It is vacant, so we can simply insert the batch as a new segment page *a*. At the second batch, $\langle 0, 4, 5 \rangle$, we again check the first level, but this time it is occupied. Thus, we must perform a merge of the keys in segment *a* and *batch 2* to produce the sequence $\langle 0, 1, 4, 5, 7, 10 \rangle$, which is split into two segments *b* $\langle 0, 1, 4 \rangle$ and *c* $\langle 5, 7, 10 \rangle$. The third batch $\langle 2, 8, 11 \rangle$ becomes a new segment *d* occupying the first level, just as the first batch. When the fourth batch $\langle 3, 6, 9 \rangle$ is inserted, it must be merged with both levels one and two, since both are occupied. This cascade of merges produces the final sorted key sequence

⁹The configuration we use is 4KB node pages and 32MB leaf pages.

¹⁰The intra-page regions in Figure 5b are not shown to scale.

¹¹We say that a pivot is *addressed* by a key if the pivot's key range contains that key.

¹²Operations such as node splits during a batch update or taking a snapshot of an existing TurtleTree may introduce additional references to a given page.

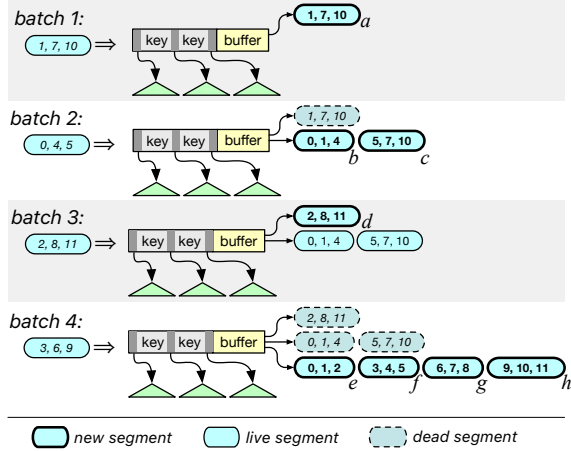


Figure 6: Insertion of Batch Updates in TurtleTrees.

$\langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 \rangle$, resulting in a new level comprised of segments $e \langle 0, 1, 2 \rangle$, $f \langle 3, 4, 5 \rangle$, $g \langle 6, 7, 8 \rangle$, and $h \langle 9, 10, 11 \rangle$.

After inserting a new batch into the buffer, if there are at least L (the size of a leaf page) buffered bytes of updates addressed to one of the pivots, then we perform a flush. This involves picking the pivot with the most buffered data, and removing L bytes of updates from the buffer in the target key range. Extracting a batch to flush may involve an in-memory merge/compaction, as the data may be spread across multiple levels. In the prior example, if we perform a flush after *batch 3*, supposing the target subtree’s key range to be $2..6$, we would need to merge data from all three segments (b, c, d) to produce the batch $\langle 2, 4, 5 \rangle$. As mentioned in §3.1, durable data in update buffer segments is “removed” by modifying the in-node active key range metadata (§3.1.2). This reduces write amplification by avoiding the need to update the segment page(s).

After a batch has been recursively flushed to a sub-tree, a split or join of the corresponding pivot may be required. All non-root nodes must be kept above a configured minimum size to remain viable. In addition, we maintain an invariant that the total amount of buffered data per node never exceeds $L \cdot (\rho - 1)$. If the buffered data does ever exceed this amount, it will be because a new batch of size L was just inserted, resulting in there being at least $L \cdot \rho$ total bytes. In this case, we will always have enough to flush a batch to some sub-tree to restore the invariant. Node joins are simpler than splits, since we can assume the invariant to hold for each node being joined. Since the key-range of two sibling nodes is guaranteed not to overlap, the invariant is preserved by simply concatenating the contents of nodes to join them. The same is not true, in general, of splits. If after splitting a node as evenly as possible, we find that the update buffer size invariant is violated for one of the new smaller nodes, we perform another flush on that node to restore it. Note that we can be sure that if this case arises, it will only be for one side, not both; otherwise, the invariant must have been false before the split. An AMQ filter per leaf page is built and flushed when batch updates are finally written to storage.

3.2.2 Queries. Point and range queries are the same for TurtleTrees as for B⁺-trees, with the addition of chasing pointers to segment page(s) in the buffer (instead of just reading its contents). We use the *activePivots* bit sets stored for each segment to narrow the search

to only those which might contain the key. Further pruning of the set of candidate pages to search is done by querying a Bloom or Quotient filter [52] associated with each segment/leaf (§4.1.2).

After finding a key in a segment, we compare its position in the leaf to the flushed upper bound for its corresponding pivot; if this position is less than the upper bound (i.e., it has been marked as flushed), we act as if the key is not present and continue the search.

Deletes are performed alongside *Put* operations as part of a batch update, by setting a key’s value to the special DELETED tombstone value. This value is preserved in leaves until the bottom of the tree is reached; at this point, the key is removed from the merged output.

3.3 TurtleTree Cost Analysis

3.3.1 Update I/O cost. Individual key updates travel from root to leaf of the tree as batch updates and flushes take place. Since there are $O(\log_2 \rho)$ levels within the buffer of each node, and the tree height is $O(\log_\rho (N/L))$, each key update is written $O(\log_2 (N/L)) = O(\log_2 \rho \cdot \log_\rho (N/L))$ times, regardless of page size or branching factor ρ .¹³ If we define the transfer size B in units equal to a single key update, the I/O cost of a key update is $O(B^{-1} \cdot \log_2 (N/L))$.

3.3.2 Query cost. Point queries (*Get*) differ in their I/O cost depending on whether or not the key is found. Here we give a worst-case bound for the general case; in practice this can be dramatically improved through the addition of per-leaf filters, as described in §4.1.2. The cost analysis here also depends on the specific I/O model used; if we are counting total page loads, as in the Disk Access Model (DAM), then a single-key (point) query must load all node pages and buffer segments along a single root-to-leaf path: $O(\log_\rho (N/L) + \frac{L}{B} \cdot \log_2 (N/L))$ or just $O(\frac{L}{B} \cdot \log_2 \frac{N}{L})$.

Like other B-tree variants, range queries of length k on TurtleTrees cost a point query (to find the starting key) plus $O(k/B)$ IOPs. There is also an in-memory, per-key cost of $\log_2 (\log_2 (N/L))$ to merge sorted runs, which is the same as a level-tiered LSM-tree. This per-key cost expression is derived via the use of a binary min-heap to select which of the $\log_2 (N/L)$ levels of the tree contains the least-ordered key.

3.3.3 Write-Memory Trade-off Efficiency. The WM tuning knob for TurtleTrees is a parameter *chi* (χ), for *checkpoint distance*, which determines the number of batches applied to a TurtleTree in-memory before flushing changed pages to storage as a new checkpoint. At $\chi = 1$, every time we perform a batch update on the tree, all modified pages must be written back out. At $\chi = 2$, we defer the writing of new/modified pages until two batches have been applied, etc. It may not be immediately obvious to the reader that χ should have any effect on the write cost of key updates. To see that it does, let us revisit the batch insertion example illustrated by Figure 6, considering the span of time (as measured by batch number) that each page is live, shown in Figure 7. The pages which appear in durable storage at a given time step are those whose lifespan overlaps a given column in the middle section (*Lifetime*) of the diagram.

Observe that when $\chi = 2$, the state of the tree is only externalized at time steps 2 and 4, so that the segments (a, d), which are live

¹³Note that in order to correctly analyze the cost of a key update, we need only consider its initial journey from root-to-leaf, since each time a key’s page is re-written after it reaches the bottom of the tree, we can amortize that cost against the incoming batch, which represents an equal number/size of keys ending their root-to-leaf journey.

Page	Lifetime				Written if ..?			
	$t=1$	$t=2$	$t=3$	$t=4 \dots$	$\chi=1$	$\chi=2$	$\chi=3$	$\chi=4$
a 1, 7, 10	█				✓	○	○	○
b 0, 1, 4		█			✓	✓	✓	○
c 5, 7, 10			█		✓	✓	✓	○
d 2, 8, 11				█	✓	○	○	○
e 0, 1, 2				█	✓	✓	✓	✓
f 3, 4, 5				█	✓	✓	✓	✓
g 6, 7, 8				█	✓	✓	✓	✓
h 9, 10, 11				█	✓	✓	✓	✓

Figure 7: The effect of checkpoint distance (χ) on write amplification. (Pages shown are from the example in Figure 6.)

only for steps 1 and 3, are never written. Also observe that when $\chi = 2$, keys 1, 7, and 10 first appear in stored pages at level two in segments *b* and *c*; i.e., they appear to an external observer (who can only look at the *externalized* or durable states of the tree) to have *skipped* the first level altogether. If one considers *batch* 3, the effect is even more pronounced: the first place keys 2, 8, and 11 appear when $\chi = 2$ is at the third level, in segments *e*, *g*, and *h*. If χ is doubled again to 4, then all keys in the example appear to skip the first two levels.

We claim that, in general, the effect of χ is that keys skip the first $\log_2 \chi$ buffer levels of the tree, which reduces the per-key write cost to:

$$O\left(\frac{1}{B} \cdot \left(\log_2 \frac{N}{L} - \log_2 \chi\right)\right) = O\left(\frac{1}{B} \cdot \log_2 \frac{N}{\chi L}\right)$$

The key limitation of TurtleTree’s χ tuning knob, and of RM+WM tuning schemes in general, is that unlike I/O operations, memory is generally a bounded quantity. This means higher χ value will spend memory that could have been used for page and filter caching. Thus the zero-sum of fixed-size memory resources is effectively turned into a read-vs-write (RW) trade-off. However, unlike RW trade-offs that affect the structure of stored data, our scheme is flexible and forgiving, since it can be changed at runtime.

4 TURTLEKV SYSTEM DESIGN

In this section we first give an overview of the system architecture and update/query paths in TurtleKV, then dive deeper into the design of two of the key ideas it implements: checkpoint distance tuning with Big MemTables, and parallel (multi-core) acceleration for TurtleTree (checkpoint) updates.

4.1 Basic Operation

TurtleKV is based on a conventional LSM-tree system architecture employing three high-level components: a write-ahead log (WAL)

to ensure durability of incoming updates, an in-memory index and buffer of recent data (MemTable), and an on-disk checkpoint for long-term storage (TurtleTree). It is implemented in 32k lines of C++ code, and is available as open-sourced software [6].

4.1.1 Update Path. Incoming key-value data are copied to an internal memory buffer which is continuously flushed in the background to the WAL. Each update is assigned a unique sequence number so that the database state can be consistently reconstructed during recovery. After the key and value bytes are copied, they are inserted into an in-memory index (the *active* MemTable) on the user thread performing the update; as soon as this completes, the update is visible to all threads in the process.

Each MemTable is limited to a maximum size $\overline{M}_w$. When a thread attempts to perform a MemTable insertion, but fails because this limit would be exceeded, the active MemTable is *finalized*, or made read-only, and a new active MemTable is created to accept future updates. The finalized MemTable is handed off to a background thread, which splits it into 1 or more leaf-page-sized batches, each of which is applied to the current checkpoint TurtleTree. The user thread must wait if the background thread is already busy processing a previous finalized MemTable. After the hand-off succeeds, it will also wait until the total number of finalized MemTables in the system is at most 2. When the background thread finishes applying all batches for a MemTable to the page cache, it hands the list of new pages to a second background thread (the *writer thread*), which writes the new checkpoint data to disk. Once the new TurtleTree pages are durable, the writer thread swaps the finalized MemTable with a reference to the new checkpoint which subsumes it, possibly unblocking any application threads that were waiting for the finalized MemTable count to decrease.

This multi-stage update pipeline allows high system resource utilization by allowing the different stages (MemTable insertion, TurtleTree updates, and checkpoint page flushing) to be performed at the same time. By keeping the amount of queued data in the pipeline to a minimum, it also provides back-pressure so that no part of the system outpaces the others, in order to avoid unbounded tail latencies.

4.1.2 Query Path. Point queries are answered by first consulting the active MemTable. If the target key is found there, the query terminates and the answer is returned to the caller. Otherwise, the ≤ 2 finalized MemTables are consulted in newest-to-oldest order. If the key is not found there either, then the most recent checkpoint is queried as described in §3.2.2.

TurtleKV supports either Bloom or Quotient filters (selected at compile-time) to optimize point queries. Each leaf page, whether at the bottom of the TurtleTree or referenced from a node buffer, has an

Table 2: Per-key, asymptotic costs (Big- O) for external memory dictionaries. Point-Query (*Get*) costs are given both for Disk Access Model (DAM) and Parallel Disk Access Model (PDAM). Scans (length k) cost a *Get* plus $\frac{k}{B}$ I/Os for all cases and models.

Operation	TurtleTree	B ⁺ -Tree	LSM (level-tiered)	LSM (size-tiered)	B ^ε -Tree	STB ^ε -Tree
<i>Put</i>	$\frac{1}{B} \cdot \log_2 \frac{N}{\chi}$	$\log_B N$	$\frac{F}{B} \cdot \log_F N$	$\frac{1}{B} \cdot \log_T N$	$\frac{1}{B^{1-\epsilon}} \cdot \log_{B^\epsilon} N$	$\frac{1}{B} \cdot \log_F N$
<i>Get</i> (DAM)	$\log_2 N$	$\log_B N$	$\log_F N \cdot \log_B N$	$T \cdot \log_T N \cdot \log_B N$	$\frac{1}{\epsilon} \cdot \log_{B^\epsilon} N$	$F \cdot \log_B N$
<i>Get</i> (PDAM)	$\log_B N$	$\log_B N$	$\log_B N$	$T \cdot \log_B N$	$\frac{1}{\epsilon} \cdot \log_{B^\epsilon} N$	$F \cdot \log_B N$

associated filter page. Whenever the query algorithm demands that a leaf be searched for a given key, the associated filter is first queried, and only if it gives a positive response is the leaf loaded/checked.

When a leaf is queried, we first attempt to pin the whole leaf page in the cache; this will fail if the page isn't in memory, without performing any I/O. If the whole leaf is not cached, then we access the first 64KB-sized slice of the page, which contains the header and trie index. From this information, we are able to determine which 4KB slice will contain the target key, if present.

4.2 Parallelization of TurtleTree Updates

4.2.1 The necessity of parallelized updates. TurtleTree checkpoint update is the most expensive operation in the TurtleKV update path by a substantial margin. Therefore it is the primary target of our optimization efforts to avoid in-memory bottlenecks, which is one of the 3 high-level requirements for TurtleKV (§2, Table 1).

To put this in context, while running YCSB workloads on 16 concurrent threads, we measured the average MemTable insertion time in TurtleKV to be approximately $1.5 \mu\text{s}$, for a combined rate across all threads of slightly over 10 million inserts per second. In our end-to-end testing, we find that this outpaces the overall load throughput of the fastest databases (SplinterDB and TurtleKV) by 2-5 $\times$. For the “small” records in our tests (8 byte keys + 120 byte values = 128 bytes), this equates to roughly 1.34 GB/s of load on the WAL, around half the bandwidth of the test SSD. For “large” records (8 byte keys + 992 byte values = 1000 bytes), MemTable insertion only becomes the bottleneck if the WAL can exceed 9.4 GB/s. Since TurtleKV’s WAL flushing implementation is easily able to match or exceed the MemTable insertion rate, we are left with checkpoint update as the potential bottleneck in the system.¹⁴

The challenge of TurtleTree updates is that they represent a potentially logarithmic amplification of work as compared to earlier pipeline stages, due to the $\log(N/L)$ levels through which update records must be merged/compacted to reach the leaf level. There is simply no way to keep up with load generated by earlier stages of the update path if we do not employ some kind of in-memory parallelization strategy.

4.2.2 Task-parallel vs Data-parallel. There are two basic approaches to this: allow concurrent updates to take place in different locations within the same tree, or somehow parallelize the work of sequentially applied batch updates. The former approach (task-parallel) is used by SplinterDB; TurtleKV takes the latter approach (data-parallel). The reason is twofold: first, there are many complications involved in implementing concurrent updates to B^+ -trees, and these are a superset of the complexities one faces when implementing concurrent B^+ -trees, a non-trivial task because of the competing concerns of root-to-leaf traversal for insertion, and leaf-to-root split/merge for re-balancing. Second, adopting a data-parallel approach affords better load balancing across processors, leading to more consistent update performance with less variability.

4.2.3 Details of parallel updates. We observe that the most CPU-intensive operations in TurtleTree batch update are the key comparisons required to merge/compact level segments when a node receives an incoming batch from its parent and flushes an outgoing

batch to a subtree. Merging multiple ordered sequences is easily done in parallel using multi-selection [31] on each sequence to determine the input and output ranges for different processors. By implementing this algorithm, we are able to scale the CPU work to multiple processors while ensuring the correctness of our implementation through simplicity of the code.

We also observe that the most memory-bandwidth expensive operation during checkpoint update is copying key-value data while serializing new compacted pages prior to writing them to storage. Our approach to optimizing this is reduce the total amount of data that must be copied by deferring all serialization until right before the checkpoint is finalized, and to serialize all leaf pages (and their corresponding filters) in parallel on different threads.

4.3 Checkpoint Distance & Big MemTables

4.3.1 Preliminaries: MemTable Implementation. The requirements for the MemTable data structure are scalability to high numbers of concurrent threads, low space overhead, ability to perform key-order scans, and CPU cache friendliness. There are several possible data structures which meet these criteria; for example, Wang et al. [58] compare Bw-Tree, OpenBW-Tree, SkipList, Masstree, B^+ -tree, and Adaptive Radix Tree (ART) implementations for in-memory, multi-threaded performance and efficiency. TurtleKV implements ARTs for this purpose, because they outperform all other data structures in this comparison, and offer low memory usage and high CPU-cache hit rate across a variety of workloads. However, TurtleKV is careful to abstract this implementation choice so that other data structures can be swapped in if desired. For comparison, RocksDB uses concurrent SkipLists as the default MemTable, WiredTiger uses a combination of B^+ -tree and SkipLists for in-memory indexing, and SplinterDB uses concurrent B^+ -trees as its MemTable and branches. The TurtleKV ART implementation is roughly the same size (lines of code) as RocksDB’s SkipList.

4.3.2 Checkpoint Distance: A Design Dilemma. TurtleKV’s write-memory (WM) tuning knob is checkpoint distance; this is the total size of the updates between two checkpoints. This can be varied at runtime to target different levels of write-optimization at the expense of higher memory footprint devoted to updates. Recall from our case study (§2.2.2), that we find there are two knobs offered by RocksDB to trade write amplification directly against memory: MemTable size $\overline{M_w}$ and MemTable count $|M_w|$. The theoretical reason behind their effectiveness is that they force the first $O(\log_f(M))$ compactions for a given key x to happen in memory, before that key is ever written to an SSTable. Varying either parameter while holding the other constant is equally effective, since $M = \overline{M_w} \cdot |M_w|$. The reason to prefer increasing $\overline{M_w}$ to $|M_w|$ as a tuning knob is that MemTables’ contribution to query cost in a RocksDB-like design is $O(|M_w| \cdot \log(\overline{M_w}))$, assuming a MemTable data structure that supports optimal logarithmic query cost.

If we apply this same logic to TurtleKV’s design, however, we encounter a dilemma. We want the on-disk structure of checkpoint TurtleTrees to be independent of checkpoint distance, so that we get a true WM tuning knob; this is possible only if we keep $\overline{M_w}$ constant and vary the number of batches $|M_w|$ in between checkpoints. Thus we have a tug-of-war between the concerns of in-memory

¹⁴Details of the WAL are omitted here for brevity; for more details, see https://github.com/mathworks/turtle_kv.

(MemTable) queries on newer data, and queries against checkpoints (TurtleTrees) in the face of varied past tuning point selection.

4.3.3 The Solution: Big MemTables. We resolve this tug-of-war by setting the size limit of a MemTable when it is created to the current checkpoint distance, and breaking up finalized MemTables into many leaf-page sized batches in order to update the TurtleTree. We do this via a least-to-greatest key-ordered scan of the MemTable. This design is called Big MemTables in TurtleKV.

Big MemTables help to optimize TurtleKV’s performance in multiple ways. First, by scaling up the size of TurtleKV’s in-memory index on recent updates, we speed up queries on “hot keys.” Second, by allowing a larger set of updates to accumulate in memory before doing a scan to break the MemTable into batches, we increase the skew in key distribution when applying batches to the checkpoint. TurtleTree batch update (§3.2.1), which flushes batches down the tree as soon as the minimum size threshold for a subtree is reached (not when the buffer fills up), transparently leverages this key skew to move data down the tree faster, so long as doing so will not hurt efficiency (i.e., so long as we preserve the property that merges are always performed on equal-sized inputs). We note that SplinterDB’s flush-then-compact policy achieves a similar effect, though it has the downside of increased space amplification due to the fact that a flushed branch may end up referenced in many different subtrees despite only containing a small amount of live data for each reference.

Finally, Big MemTables further motivate our selection of ARTs as MemTable data structure. Because the structure of radix trees allow for segments of indexed keys to be shared, it is possible for the per-key overhead of ARTs to fall below the average key length [43]; this efficiency is often increased as the total set of keys in an ART scales. Thus by increasing $\overline{M}_w$ and then splitting buffered updates into key-skewed batches, we not only play to the strengths of TurtleTrees, but to those of ARTs as well.

5 EVALUATION

In this section, we explore the end-to-end performance of TurtleKV, comparing it to WiredTiger, RocksDB, and SplinterDB. We provide data to support the claims we make about TurtleKV and its novel data structure TurtleTrees, and our claims about the necessity of write-memory (WM) trade-off tuning more generally.

5.1 Experimental Setup

5.1.1 Hardware setup. Experiments were run on an AMD Ryzen Threadripper 7970x workstation (32 cores/64 threads), with 128 GB of DDR5-6000 registered ECC memory, running Linux 6.3. The SSD used was an Intel P4800x 1.5TB, with 512 byte LBA configured, formatted with XFS. The P4800x is a data center grade Optane device supporting PCIe 3.0, rated for 2.4 GB/s read bandwidth and 2 GB/s write (550k/500k IOPs respectively).

5.1.2 Software/Benchmark Configuration. We use the Yahoo Cloud Serving Benchmark (YCSB) [25], comparing TurtleKV to other state-of-the-art key-value engines, each representing a different optimization target: WiredTiger, a read-optimized B⁺-tree-based engine; RocksDB, a write-optimized and widely used LSM-tree-based system; and SplinterDB, a state-of-the-art hybrid-optimized key-value store. Each YCSB run inserts 400M records comprised of 8 byte keys and 120 byte values (Load), then performs 150M operations

comprised of 50% updates/50% gets (workload A), another 150M operations comprised of 5% updates/95% gets (workload B), 150M gets (workload C), 15M operations comprised of 95% scans (length ≤ 100 keys)/5% updates (workload E), and then a final 150M operations comprised of 50% read-modify-writes/50% gets (workload F). The total size of the initial data set is 48GB; to evaluate both out-of-cache and in-cache performance, we run the benchmark twice for each database, once with cache size 16GB ($\frac{1}{3}$ of data size) and again with 48GB cache (large enough to fit the raw data).

5.1.3 Database Configurations. The configuration used for each database is informed by the case studies described in §2.2. For WiredTiger, eviction dirty target is set to half the cache size and eviction dirty trigger to 95% of the cache; transaction-level logging is enabled, and automatic checkpoints are disabled. For RocksDB, parallelism and maximum parallel sub-compactions are set to the processor count (64); write buffer size is 64MB. RocksDB’s block cache is enabled (instead of the kernel’s file cache) to accurately track memory usage; direct I/O is enabled, which we observe to be slightly faster. SplinterDB is configured with mostly default values. We set the maximum key size to 8 bytes to match our test workload, and configure the number of background threads to 2 for memtable tasks, 24 for normal tasks, following the guidance of the SplinterDB tuning guide [5].

TurtleKV is configured to use 32MB leaf pages, 20 bits per key for per-leaf quotient filters (SplinterDB’s default was left unchanged at 26). Checkpoint distance was configured in two different ways: we selected 4 different values for each cache size (128MB, 256MB, 512MB, and 768MB for 16GB cache; 512MB, 1GB, 1.5GB, and 3GB for 48GB cache) to run TurtleKV *statically tuned* (i.e., all YCSB workloads run with the same setting). We also selected, through trial and error, a “known good” setting for checkpoint distance for each workload and cache size combination: for 16GB cache, Load $\rightarrow$ 512MB, A/B $\rightarrow$ 768MB, C $\rightarrow$ 32MB, E $\rightarrow$ 192MB, and F $\rightarrow$ 640MB. For 48GB cache, Load/A/B $\rightarrow$ 3GB, C $\rightarrow$ 32MB, E $\rightarrow$ 256MB, and F $\rightarrow$ 2GB. We then configure the test runner to run TurtleKV *dynamically tuned* by switching to the “known good” value at the start of each workload. Although this is probably not a practical technique for real-world use, we performed this test to verify the potential of TurtleKV’s WM tuning knob. All comparisons with the other databases show TurtleKV’s dynamic tuning mode.

5.2 Results & Discussion

5.2.1 YCSB Throughput: TurtleKV vs Others. Figure 8 shows the end-to-end throughput of each YCSB workload, for dynamically-tuned TurtleKV and all other databases.

TurtleKV performed excellently in the high-memory configuration across all workloads (Figures 8a, 8b), exceeded only in scan-dominated workload E by WiredTiger. WiredTiger’s B⁺-tree demonstrates a clear advantage for scans in both cache configurations, since it does not require any in-memory merging, unlike LSM-trees, STB^c-trees, and TurtleTrees.

For cache-constrained write-heavy workloads (Figure 8c), SplinterDB was the highest performer, which agrees with our case studies on WM tuning (§2.2). In the cache-constrained read-heavy workloads (Figure 8d), TurtleKV was able to match or exceed the point query throughput of the other write-optimized systems (RocksDB and SplinterDB), which is expected because TurtleTree’s level-tiered

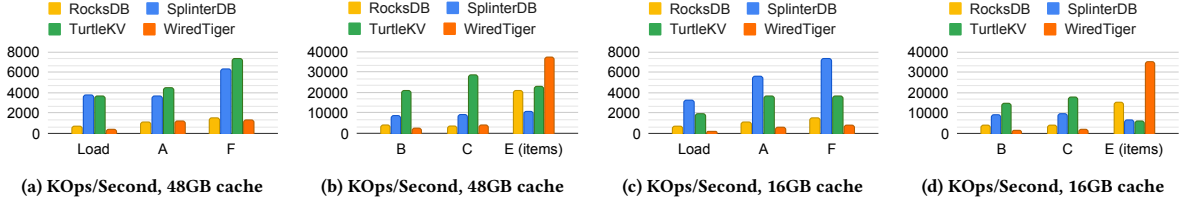


Figure 8: YCSB, $N=400M \times 128b$

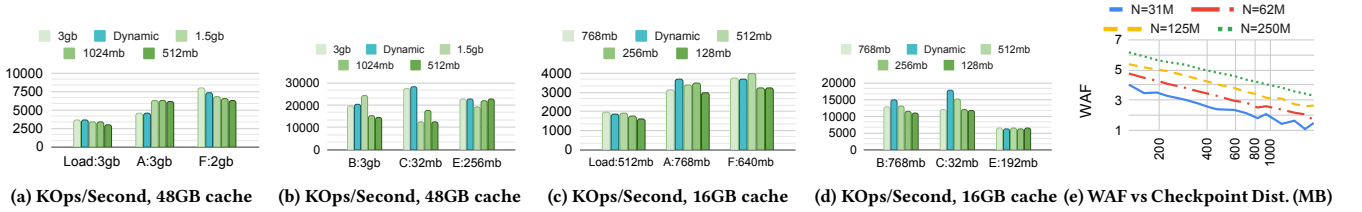


Figure 9: TurtleKV WM Tuning Sensitivity

compaction policy is more read-optimized than the lazy compaction of SplinterDB. TurtleKV’s performance in workload E (scans) shows room for improvement; we believe the low scan rate is due to cache pressure from the active (unflushed/uncompacted) MemTable left over from previous workloads. This data supports the idea that TurtleKV’s scan performance in cache-limited configurations could be improved by implementing a mechanism to detect query-driven cache pressure and respond by flushing the current MemTable to the checkpoint TurtleTree to free up memory.

5.2.2 Static vs Dynamic Tuning. Figure 9 (a-d) shows TurtleKV’s sensitivity to checkpoint distance tuning parameter selection across YCSB workloads. Particularly noteworthy is the benefit of setting a lower checkpoint distance for point-query dominated workloads B and C (9d, 9b). The ability to give explicit direction to the system as to how it should use memory helps TurtleKV to achieve much higher performance in these workloads than the other systems (8d, 8b), including TurtleKV itself when tuned statically.

Figure 9e demonstrates our claim that the write-optimization benefit of WM tuning in TurtleTrees scales independently of total dictionary size N . The higher variation in the smallest dataset ($N=31M$) is because this scenario is more sensitive to small differences in the number of batches flushed down the tree; the leaf page size L is configured to 32MB, which is a larger percentage of the overall data size at smaller scale.

5.2.3 YCSB Latency Percentiles. Figure 10 shows the per-operation latency distributions for the YCSB tests. A key result in this data is that TurtleKV demonstrates superior tail latencies (P99.9) for both inserts and updates, supporting the effectiveness of TurtleKV’s update pipeline and multi-processor approach to checkpoint updates. TurtleKV falls behind SplinterDB in low-cache, insert-heavy workloads due to high P100 during checkpoint pipeline stalls.

While latencies for queries are generally good, TurtleKV does present somewhat elevated tail latencies for point queries with both high and low cache sizes, and for small-cache scans.

5.2.4 Memory Usage over Time. Figure 11 shows memory usage in the benchmark process for each database over the course of the YCSB workloads. TurtleKV and SplinterDB both quickly make use of available memory during the Load; afterwards, both generally stayed below their allocated limit. Note, because the load generator is in the same process as the key-value stores being tested, and because we used pre-recorded workload trace files that are buffered in memory, the overall memory footprint is somewhat higher than the configured cache limit. Both RocksDB and WiredTiger did not make use of all available headroom during write-heavy workloads. RocksDB did slightly better than TurtleKV and SplinterDB at staying within the configured limit. WiredTiger presented the highest memory overhead during the benchmark.

5.2.5 CPU Efficiency for In-Cache. Figure 12 shows the CPU cache miss rate for each workload of YCSB. TurtleKV is particularly cache-friendly in query-heavy workloads C and E, due to its careful attention to page layout (cache-oblivious trie indexes at the head of each leaf page) and use of ARTs. In mixed workloads A and F, TurtleKV performs similarly in this metric to leaders RocksDB and WiredTiger. For the Load workload, CPU cache miss rate is elevated due to the shuffling effect of data-parallel batch compactions while updating the TurtleTree checkpoint; this is comparable with SplinterDB’s cache miss rate for the Load phase.

6 RELATED WORK

The theoretical trade-off space constraining external memory indexing is characterized by Brodal and Fagerberg [15] for comparison-based dictionaries, and by Conway et al. [24] for hash-based dictionaries. The key insights of these works are echoed and elaborated in practical terms by the RUM Conjecture [10].

Recent efforts to explore trade-offs in LSM-tree based systems include Monkey [27], a key-value store which optimizes queries through an analytically optimal allocation of memory to Bloom

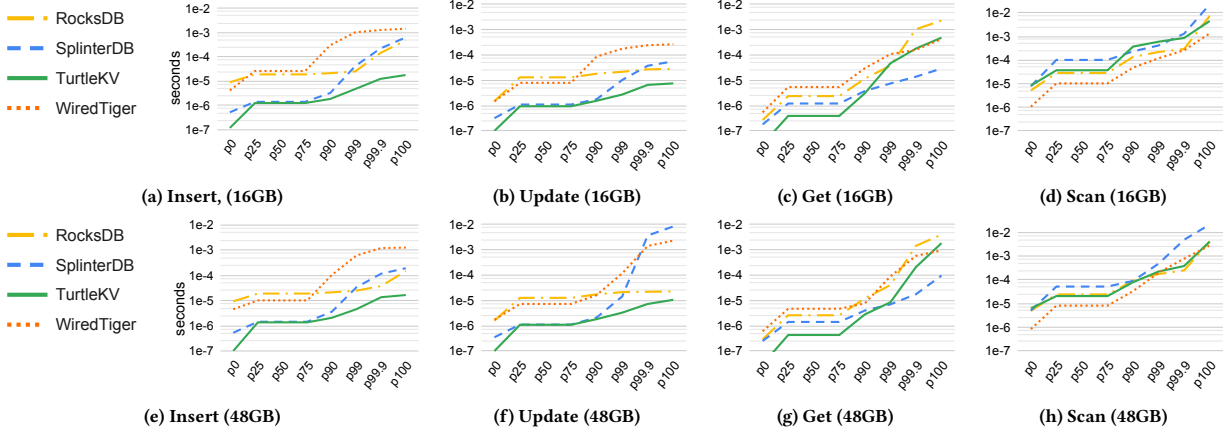


Figure 10: YCSB, Per-Operation Latency Distributions for 16GB and 48GB Cache Sizes

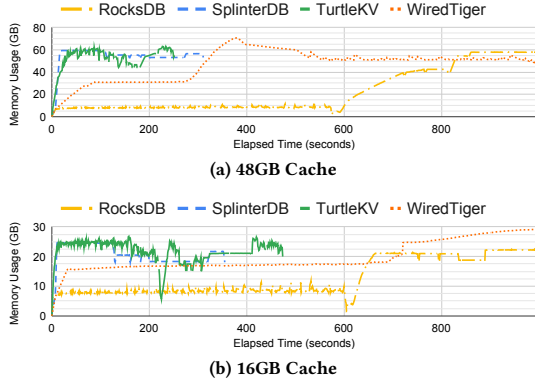


Figure 11: Memory Usage over Time During YCSB

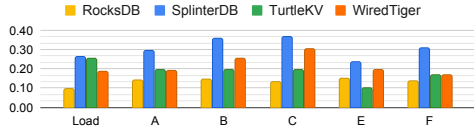


Figure 12: CPU Cache Miss Rate, 48GB cache

filters at various levels of the tree; Dostoevsky [29], which further improves space-time trade-offs for LSM-based systems by varying compaction policy from lazy at lower levels to greedy at higher ones; Wacky [30], which introduces adaptive level sizing and finer-grained control over compaction policy to even better optimize the trade-off curve; and Cosine [20], which builds on the previous three and uses machine learning to optimize data structure design for specific cloud-based key-value storage workloads. We note that Dostoevsky and Wacky in particular share some thematic similarity to TurtleKV’s scheme of optimizing writes without harming future query performance by deferring on-disk compaction of recent updates and compacting older data more aggressively. Endure [36] tunes size ratio, memory allocations, and compaction policy to maximize throughput under workload uncertainty.

Notable work in extending the trade-off tuning potential of B-tree based stores includes the Bw-Tree [46, 58], a log-structured B-Tree which uses lock-free techniques to achieve low overheads for in-memory operations in high concurrency settings; Bf-Trees, a similar approach which introduces the innovation of caching

mini-pages using a circular buffer to reduce I/O amplification while minimizing the overhead of cache maintenance; and the Bw^e-Tree [57], which describes many practical improvements to the Bw-Tree design to optimize for fast SSDs. LSB-Trees [41] propose an alternate mechanism for storing B-Tree deltas using a combination of physical and logical logging. The relationship of Log-Structured B-Trees to the overall continuum of trade-offs in the external memory dictionary design space is explored by Idreos et al. [38].

Several recent works address key-value engine design in the face of very fast modern storage devices. SplinterDB [22] and Mapped SplinterDB [23] introduce innovations in the design of external memory dictionaries, extending B^e-Trees to STB^e-Trees by applying ideas from size-tiered LSM-trees to achieve high update efficiency without excessively penalizing point queries. FASTER [18, 19] combines log-structured storage with in-memory concurrent hash tables to scale point-query-only based workloads to modern SSDs, but lacks first-class support for range operations. KVell [44], later extended to KVell+ [45], achieves high performance on modern SSDs using a *shared-nothing* design based on independent sharded indices maintained on separate threads. KVell only maintains key-based indices in memory (not in storage), trading scalability and recovery time for lower space-/write-amplification.

7 CONCLUSION & FUTURE WORK

With TurtleKV, we introduce a novel approach to read and write optimization that dynamically trades memory to establish a desired trade-off point *on demand*. The TurtleTree design and its accompanying tuning knob, checkpoint distance (χ), achieves this property by creating an unbiased on-disk structure whose read and write I/O cost can be directly tuned via dynamic memory allocation without requiring external data structures that permanently consume available memory resources.

This dynamic approach to performance optimization opens many paths for future research and improvement of key-value storage techniques. These include extending the dynamic range of χ -based tuning by further optimizing in-memory checkpoint tree updates, reducing memory requirements through the use of streaming compaction and other techniques, and exploring more of the B^{e+}-Tree design space to discover data structures beyond the TurtleTree.

REFERENCES

- [1] 2022. Revisiting B+-Tree vs. LSM-tree. <https://www.usenix.org/publications/loginonline/revisit-b-tree-vs-lsm-tree-upon-arrival-modern-storage-hardware-built>.
- [2] 2022. Small Datum: The Impact of Max_background_jobs on Write Throughput for RocksDB. <https://smalldatum.blogspot.com/2022/07/the-impact-of-maxbackgroundjobs-on.html>.
- [3] 2024. RocksDB | A Persistent Key-Value Store. <http://rocksdb.org/>.
- [4] 2025. RocksDB Tuning Guide. <https://github.com/facebook/rocksdb/wiki/RocksDB-Tuning-Guide>.
- [5] 2025. SplinterDB. <https://splinterdb.org/>.
- [6] 2025. TurtleKV | High Performance, Dynamically Tuned Embedded Key-Value Storage Engine. https://github.com/mathworks/turtle_kv.
- [7] 2025. WiredTiger: WiredTiger Developer Site. <https://source.wiredtiger.com/>.
- [8] Remo Andreoli and Tommaso Cucinotta. 2023. Inducing Huge Tail Latency on a MongoDB Deployment. In *2023 IEEE International Conference on Cloud Engineering (IC2E)*. 107–112. <https://doi.org/10.1109/IC2E59103.2023.00020>
- [9] Saman Ashkiani, Shengren Li, Martin Farach-Colton, Nina Amenta, and John D. Owens. 2018. GPU LSM: A Dynamic Dictionary Data Structure for the GPU. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, Vancouver, BC, 430–440. <https://doi.org/10.1109/IPDPS.2018.00053>
- [10] Manos Athanassoulis, Michael S Kester, Lukas M Maas, Radu Stoica, Stratos Idreos, Anastasia Ailamaki, and Mark Callaghan. 2016. Designing Access Methods: The RUM Conjecture. *EDBT* (2016).
- [11] R Bayer. 1970. Organization and Maintenance of Large Ordered Indices. *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control* (1970).
- [12] Michael A. Bender, Martin Farach-Colton, Jeremy T. Fineman, Yonatan R. Fogel, Bradley C. Kuszmaul, and Jelani Nelson. 2007. Cache-Oblivious Streaming B-trees. In *Proceedings of the Nineteenth Annual ACM Symposium on Parallel Algorithms and Architectures*. ACM, San Diego California USA, 81–92. <https://doi.org/10.1145/1248377.1248393>
- [13] Michael A Bender, Martin Farach-Colton, William Jannen, Rob Johnson, Bradley C Kuszmaul, Donald E Porter, Jun Yuan, and Yang Zhan. 2015. An Introduction to B ϵ -trees and Write-Optimization. *login; magazine* 40, 5 (Sept. 2015).
- [14] Burton H. Bloom. 1970. Space/Time Trade-Offs in Hash Coding with Allowable Errors. *Commun. ACM* 13, 7 (July 1970), 422–426. <https://doi.org/10.1145/362686.362692>
- [15] Gerth Stølting Brodal and Rolf Fagerberg. 2003. Lower Bounds for External Memory Dictionaries. *ACM-SIAM Symposium on Discrete Algorithms (SODA)* 3 (2003).
- [16] Zhichao Cao, Siying Dong, Sagar Vemuri, and David H C Du. 2020. Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook FAST (2020).
- [17] Hokeun Cha, Xiangpeng Hao, Tianzheng Wang, Huanchen Zhang, Aditya Akella, and Xiangyao Yu. 2023. *Blink* -Hash: An Adaptive Hybrid Index for In-Memory Time-Series Databases. *Proceedings of the VLDB Endowment* 16, 6 (Feb. 2023), 1235–1248. <https://doi.org/10.14778/3583140.3583143>
- [18] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *Proceedings of the 2018 International Conference on Management of Data*. ACM, Houston TX USA, 275–290. <https://doi.org/10.1145/3183713.3196898>
- [19] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: An Embedded Concurrent Key-Value Store for State Management. *Proceedings of the VLDB Endowment* 11, 12 (Aug. 2018), 1930–1933. <https://doi.org/10.14778/3229863.3236227>
- [20] Subarna Chatterjee, Meena Jagadeesan, Wilson Qin, and Stratos Idreos. 2021. Cosine: A Cloud-Cost Optimized Self-Designing Key-Value Storage Engine. *Proceedings of the VLDB Endowment* 15, 1 (Sept. 2021), 112–126. <https://doi.org/10.14778/3485450.3485461>
- [21] Douglas Comer. 1979. Ubiquitous B-Tree. *Comput. Surveys* 11, 2 (June 1979), 121–137. <https://doi.org/10.1145/356770.356776>
- [22] Alexander Conway, Vijay Chidambaram, Martin Farach-Colton, and Rob Johnson. 2020. SplinterDB: Closing the Bandwidth Gap for NVMe Key-Value Stores. *2020 USENIX Annual Technical Conference (USENIX ATC 20)* (2020).
- [23] Alex Conway, Martin Farach-Colton, and Rob Johnson. 2023. SplinterDB and Maplets: Improving the Tradeoffs in Key-Value Store Compaction Policy. *Proceedings of the ACM on Management of Data* 1, 1 (May 2023), 1–27. <https://doi.org/10.1145/3588726>
- [24] Alex Conway, Martin Farach-Colton, and Philip Shilane. 2018. Optimal Hashing in External Memory. <https://doi.org/10.48550/arXiv.1805.09423> [cs]
- [25] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing*. ACM, Indianapolis Indiana USA, 143–154. <https://doi.org/10.1145/1807128.1807152>
- [26] Databricks. 2025. Extract Transform Load (ETL). <https://www.databricks.com/discover/etl>. Retrieved August, 2025.
- [27] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal Navigable Key-Value Store. In *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, Chicago Illinois USA, 79–94. <https://doi.org/10.1145/3035918.3064054>
- [28] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2018. Optimal Bloom Filters and Adaptive Merging for LSM-Trees. *ACM Transactions on Database Systems* 43, 4 (Dec. 2018), 1–48. <https://doi.org/10.1145/3276980>
- [29] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging. In *Proceedings of the 2018 International Conference on Management of Data*. ACM, Houston TX USA, 505–520. <https://doi.org/10.1145/3183713.3196927>
- [30] Niv Dayan and Stratos Idreos. 2019. The Log-Structured Merge-Bush and the Wacky Continuum. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, Amsterdam Netherlands, 449–466. <https://doi.org/10.1145/3299869.3319903>
- [31] Narsingh Deo, Amit Jain, and Muralidhar Medidi. 1994. An Optimal Parallel Algorithm for Merging Using Multiselection. *Inform. Process. Lett.* 50, 2 (April 1994), 81–87. [https://doi.org/10.1016/0020-0190\(94\)00009-3](https://doi.org/10.1016/0020-0190(94)00009-3)
- [32] Shaker H. Ali El-Sappagh, Abdeltawab M. Ahmed Hendawi, and Ali Hamed El Bastawissy. 2011. A Proposed Model for Data Warehouse ETL Processes. *Journal of King Saud University - Computer and Information Sciences* 23, 2 (July 2011), 91–104. <https://doi.org/10.1016/j.jksuci.2011.05.005>
- [33] Alexandra Fedorova, Craig Mustard, Ivan Beschastnikh, Julia Rubin, Augustine Wong, Svetozar Mucic, and Louis Ye. 2018. Performance Comprehension at WiredTiger. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Lake Buena Vista FL USA, 83–94. <https://doi.org/10.1145/3236024.3236081>
- [34] Eran Gilad, Edward Bortnikov, Anastasia Braginsky, Yonatan Gottesman, Eshcar Hillel, Idit Keidar, Nurit Moscovici, and Rana Shahout. 2020. EvenDB: Optimizing Key-Value Storage for Spatial Locality. In *Proceedings of the Fifteenth European Conference on Computer Systems*. ACM, Heraklion Greece, 1–16. <https://doi.org/10.1145/3342195.3387523>
- [35] Xiangpeng Hao and Badrish Chandramouli. 2024. Bf-Tree: A Modern Read-Write-Optimized Concurrent Larger-Than-Memory Range Index. *Proceedings of the VLDB Endowment* 17, 11 (July 2024), 3442–3455. <https://doi.org/10.14778/3681954.3682012>
- [36] Andy Huynh, Harshal A. Chaudhari, Evimaria Terzi, and Manos Athanassoulis. 2022. Endure: a robust tuning paradigm for LSM trees under workload uncertainty. *Proc. VLDB Endow.* 15, 8 (April 2022), 1605–1618. <https://doi.org/10.14778/3529337.3529345>
- [37] Stratos Idreos and Mark Callaghan. 2020. Key-Value Storage Engines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 2667–2672. <https://doi.org/10.1145/3318464.3383133>
- [38] Stratos Idreos, Niv Dayan, Wilson Qin, Mali Akmanalp, Sophie Hilgard, Andrew Ross, James Lennon, Varun Jain, Harshita Gupta, David Li, and Zichen Zhu. 2019. Design Continuums and the Path Toward Self-Designing Key-Value Stores That Know and Learn. *Biennial Conference on Innovative Data Systems Research (CIDR)* (2019).
- [39] Meena Jagadeesan and Garrett Tanzer. 2020. From Worst-Case to Average-Case Analysis: Accurate Latency Predictions for Key-Value Storage Engines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. ACM, Portland OR USA, 2853–2855. <https://doi.org/10.1145/3318464.3384408>
- [40] Christopher Jermaine, Anindya Datta, and Edward Omiecinski. 1999. A Novel Index Supporting High Volume Data Warehouse Insertions. *Proceedings of the 25th International Conference on Very Large Data Bases* (1999).
- [41] Bo-kyeong Kim and Dong-Ho Lee. 2015. LSB-Tree: A Log-Structured B-Tree Index Structure for NAND Flash SSDs. *Design Automation for Embedded Systems* 19, 1-2 (March 2015), 77–100. <https://doi.org/10.1007/s10617-014-9139-4>
- [42] Prateek Kumar and Veena Gaded. 2019. Value Proposition and ETL Process in Big Data Environment. *International Journal of Cloud Computing* (2019).
- [43] V. Leis, Alfons Kemper, and T. Neumann. 2013. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, Brisbane, QLD, 38–49. <https://doi.org/10.1109/ICDE.2013.6544812>
- [44] Baptiste Lepers, Oana Balmau, Karan Gupta, and Willy Zwaenepoel. 2019. KVell: The Design and Implementation of a Fast Persistent Key-Value Store. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. ACM, Huntsville Ontario Canada, 447–461. <https://doi.org/10.1145/3341301.3359628>
- [45] Baptiste Lepers, Oana Balmau, Karan Gupta, and Willy Zwaenepoel. 2020. KVell+: Snapshot Isolation without Snapshots. *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)* (2020).
- [46] J. J. Levandoski, D. B. Lomet, and S. Sengupta. 2013. The Bw-Tree: A B-tree for New Hardware Platforms. In *2013 IEEE 29th International Conference on Data*

- Engineering (ICDE)*. IEEE, Brisbane, QLD, 302–313. <https://doi.org/10.1109/ICDE.2013.6544834>
- [47] Hyeontaek Lim, Bin Fan, David G Andersen, and Michael Kaminsky. 2011. SILT: A Memory-Efficient, High-Performance Key-Value Store. *SOSP* (2011).
 - [48] Chen Luo and Michael J. Carey. 2020. LSM-based Storage Techniques: A Survey. *The VLDB Journal* 29, 1 (Jan. 2020), 393–418. <https://doi.org/10.1007/s00778-019-00555-y> arXiv:1812.07527 [cs]
 - [49] Rajendrani Mukherjee and Pragma Kar. 2017. A Comparative Review of Data Warehousing ETL Tools with New Trends and Industry Insight. In *2017 IEEE 7th International Advance Computing Conference (IACC)*. 943–948. <https://doi.org/10.1109/IACC.2017.0192>
 - [50] Trong-Dat Nguyen and Sang-Won Lee. 2016. I/O Characteristics of MongoDB and Trim-Based Optimization in Flash SSDs. In *Proceedings of the Sixth International Conference on Emerging Databases: Technologies, Applications, and Theory*. ACM, Jeju Republic of Korea, 139–144. <https://doi.org/10.1145/3007818.3007844>
 - [51] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. 1996. The Log-Structured Merge-Tree (LSM-tree). *Acta Informatica* 33, 4 (June 1996), 351–385. <https://doi.org/10.1007/s002360050048>
 - [52] Prashant Pandey, Alex Conway, Joe Durie, Michael A. Bender, Martin Farach-Colton, and Rob Johnson. 2021. Vector Quotient Filters: Overcoming the Time/Space Trade-Off in Filter Design. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 1386–1399. <https://doi.org/10.1145/3448016.3452841>
 - [53] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang, Justin Meza, and Kaushik Veeraraghavan. 2015. Gorilla: A Fast, Scalable, in-Memory Time Series Database. *Proceedings of the VLDB Endowment* 8, 12 (Aug. 2015), 1816–1827. <https://doi.org/10.14778/2824032.2824078>
 - [54] Kai Ren, Qing Zheng, Joy Arulraj, and Garth Gibson. 2017. SlimDB: A Space-Efficient Key-Value Storage Engine for Semi-Sorted Data. *Proceedings of the VLDB Endowment* 10, 13 (Sept. 2017), 2037–2048. <https://doi.org/10.14778/3151106.3151108>
 - [55] Franco Solleza, Andrew Crotty, Suman Karumuri, Nesime Tatbul, and Stan Zdonik. 2022. Mach: A Pluggable Metrics Storage Engine for the Age of Observability. *CIDR* (2022).
 - [56] Hao Wang, Jiaxin Ou, Ming Zhao, Sheng Qiu, Yizheng Jiao, Yi Wang, Qizhong Mao, Zhengyu Yang, Yang Liu, Jianshun Zhang, Jianyang Hu, Jingwei Zhang, Jinrui Liu, Jiaqiang Chen, Yong Shen, Lixun Cao, Heng Zhang, Hongde Li, Ming Li, Yue Ma, Lei Zhang, Jian Liu, Guanghui Zhang, Fei Liu, and Jianjun Chen. 2024. LavaStore: ByteDance’s Purpose-Built, High-Performance, Cost-Effective Local Storage Engine for Cloud Services. *Proceedings of the VLDB Endowment* 17, 12 (Aug. 2024), 3799–3812. <https://doi.org/10.14778/3685800.3685807>
 - [57] Rui Wang, Xinjun Yang, Feifei Li, David B. Lomet, Xin Liu, Panfeng Zhou, Yongxiang Chen, David Zhang, Jingren Zhou, and Jiesheng Wu. 2024. Bw^e-Tree: An Evolution of Bw-tree on Fast Storage. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, Utrecht, Netherlands, 5266–5279. <https://doi.org/10.1109/ICDE60146.2024.00396>
 - [58] Ziqi Wang, Andrew Pavlo, Hyeontaek Lim, Viktor Leis, Huanchen Zhang, Michael Kaminsky, and David G. Andersen. 2018. Building a Bw-Tree Takes More Than Just Buzz Words. In *Proceedings of the 2018 International Conference on Management of Data*. ACM, Houston TX USA, 473–488. <https://doi.org/10.1145/3183713.3196895>
 - [59] Ke Yi. 2012. Dynamic Indexability and the Optimality of B-Trees. *J. ACM* 59, 4 (Aug. 2012), 1–19. <https://doi.org/10.1145/2339123.2339129>